05 — Algorithm Patterns for Q4

Q4 needs one *idea*. The good news: on the GCA it's almost always one of the five patterns below. (And remember: Q4 is your **bonus** question — 3/4 already passes. Don't panic here.)

---

0. When is brute force OK?

"Brute force" = the obvious, simple way (often a double loop), even if it's slow. **Whether it's fast enough depends on the input size** `n` the problem promises. Rough rule for Python (it does very roughly 10–100 million simple operations per second):

Input size n	Safe approach	Meaning
n ≤ 1,000	$O(n^2)$ — a double loop is fine	**just brute force it!**
n ≤ 100,000	O(n log n) or O(n)	sort once, or a single pass with dict/set
n ≤ 10,000,000	O(n) only	a single pass, no nesting

> **"$O(n^2)$" just means "a loop inside a loop"** — for `n` items you do about `n × n` steps. If the problem says `n ≤ 1000`, that's a million steps — instant. So **check the limits first**: if they're small, don't be clever, be correct. A slow-but-right answer scores; a clever-but-broken one scores zero.

---

1. Dictionary / Set lookups (the most common Q4 — see file 03)

"Have I seen X before?", counting, pairs summing to a target. This is pattern #1 for a reason — reach for it first. Full explanations and code are in file 03.

Real GCA problem — `sumOfTwo(a, b, v)`: do arrays `a` and `b` contain one element each that together sum to `v`? `([1,2,3], [10,20,4], 5)` → True (1 + 4).

```
def sum_of_two(a, b, v):
    seen = set(a)                # everything in a, for instant lookup
    for x in b:
        if v - x in seen:        # is x's partner in a?
            return True
    return False
```

Why a set: checking `v - x in a_list` for every `x` is $O(n^2)$; `in a_set` is instant → O(n).

---

2. Prefix sums — add up any range instantly

The problem it solves: "what's the sum of the items between position i and j?" — asked many times, or over many different ranges. Re-adding each range is slow. A **prefix-sum array** stores running totals so any range sum becomes one subtraction.

```
nums = [3, 1, 4, 1, 5]
prefix = [0]                          # prefix[k] = sum of the FIRST k items
for x in nums:
    prefix.append(prefix[-1] + x)     # each entry = previous total + this item
# prefix is now [0, 3, 4, 8, 9, 14]
# sum of nums[i..j] inclusive  ==  prefix[j+1] - prefix[i]
```

Why it works: `prefix[j+1]` is "everything up to j" and `prefix[i]` is "everything before i." Subtract, and only the middle chunk survives. Sum of `nums[1..3]` = `prefix[4] - prefix[1]` = `9 - 3` = `6` (which is `1+4+1` ✓). Each range answer is now ONE subtraction instead of a loop.

Count subarrays summing to k (classic Q4 — prefix sums + a dict together):

```
def subarray_sum_count(nums, k):
    count, running, seen = 0, 0, {0: 1}   # seen = {running total : how many
times}
    for x in nums:
        running += x                       # running total up to here
        count += seen.get(running - k, 0)  # how many earlier totals make a gap of
k?
        seen[running] = seen.get(running, 0) + 1
    return count
```

Idea: if two running totals differ by exactly `k`, the chunk between them sums to `k`. So we remember every running total we've seen and ask "have I seen one that's `k` smaller?"

Real GCA problem — `countSubarraysSummingToK(nums, k)`: how many contiguous subarrays add up to `k`? `([1,1,1], 2)` → 2 (positions 0–1 and 1–2).

```
def count_subarrays(nums, k):
    count = 0
    running = 0
    seen = {0: 1}                      # a running sum of 0 has been "seen" once
(empty start)
    for x in nums:
        running += x
        count += seen.get(running - k, 0)   # earlier sums that leave a gap of k
        seen[running] = seen.get(running, 0) + 1
    return count
```

Trace `([1,1,1], 2)`: running goes 1, 2, 3. At running 2, `2-2=0` is in seen (once) → count 1. At running 3, `3-2=1` is in seen (once) → count 2. Answer 2 ✓

---

3. Two pointers — sorted data, or scanning from both ends

Idea: instead of one moving index, use **two** — often one at each end — and move them toward each other based on what you find.

```
# Is there a pair that sums to target, in a SORTED list?
def has_pair(nums, target):
    lo, hi = 0, len(nums) - 1        # one finger at the start, one at the end
    while lo < hi:
        s = nums[lo] + nums[hi]
        if s == target: return True
        if s < target:  lo += 1      # sum too small → grow it → move left finger
right
        else:           hi -= 1      # sum too big   → shrink it → move right
finger left
    return False
```

Why it needs a sorted list: because it's sorted, moving `lo` right *always* increases the sum and moving `hi` left *always* decreases it — so each step is a confident decision. This turns a double loop ($O(n^2)$) into a single sweep ($O(n)$). Also used for: reversing in place, merging two sorted lists, "container"-style problems.

Real GCA problem — `hasPairWithSum(nums, target)`: is there any pair adding to `target`? `([8,1,5,3], 6)` → True (1 + 5).

```
def has_pair_with_sum(nums, target):
    nums = sorted(nums)                 # two pointers NEED a sorted list
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        s = nums[lo] + nums[hi]
        if s == target:
            return True
        if s < target:
            lo += 1                     # too small → move up from the bottom
        else:
            hi -= 1                     # too big → move down from the top
    return False
```

Trace sorted `[1,3,5,8]`, target 6: 1+8=9 too big → hi--; 1+5=6 → True ✓ (You could also do this with a set, Two-Sum style — both are fine.)

---

4. Sliding window — best/longest of a CONTIGUOUS chunk

Idea: keep a "window" (a start and end) over a contiguous stretch. **Grow** it on the right; when it breaks a rule, **shrink** it from the left. Two fingers again, both moving rightward, never backward — so it's fast.

```
def longest_unique_substring(s):
    seen = set()                        # characters currently inside the window
    left = best = 0
```

```
    for right in range(len(s)):       # right edge sweeps across the string
        while s[right] in seen:       # new char already inside? window is invalid
            seen.remove(s[left])      # shrink from the left until it's valid
again
            left += 1
        seen.add(s[right])            # now safe to include the new char
        best = max(best, right - left + 1)   # window length = right - left + 1
    return best
```

Trace `"abca"`: window grows `a`, `ab`, `abc` (best 3); at the second `a`, shrink off the left `a`, then include the new one → window `bca` (still 3). Answer 3. Use this whenever you see **"longest / shortest contiguous substring or subarray with property X."**

Fixed-size window (size k): slide it along — add the element entering on the right, remove the one leaving on the left, update your best. No inner `while` needed.

Real GCA problem — `maxSumOfWindow(nums, k)`: the biggest sum of any `k` consecutive items. `([2,1,5,1,3,2], 3)` → 9 (5+1+3).

```
def max_sum_of_window(nums, k):
    window = sum(nums[:k])              # sum of the FIRST k
    best = window
    for i in range(k, len(nums)):
        window += nums[i] - nums[i - k] # add the new one, drop the one that left
        best = max(best, window)
    return best
```

Why slide instead of re-summing: re-adding k numbers each step is O(n·k); sliding is O(n). The window sum only changes by "one in, one out."

---

5. Sort first, then it's easy

Many problems collapse once the data is in order:

- **min difference** between any two values → sort, then only check neighbors
- **"can we pair everyone up"** → sort both lists, compare position by position
- **k largest** → `sorted(nums, reverse=True)[:k]`
- **custom order** → `sorted(items, key=lambda x: (x[1], -x[0]))` (read: "sort by the 2nd field; break ties by the 1st field, descending")

Sorting costs O(n log n) — cheap — and often makes the rest trivial. If you're stuck, ask: *"would this be easier if the data were sorted?"*

Real GCA problem — `minDifference(nums)`: smallest gap between any two values. `[4,9,1,32,13]` → 3 (between 1 and 4).

```
def min_difference(nums):
    nums = sorted(nums)                 # after sorting, the closest pair are
NEIGHBOURS
    best = nums[1] - nums[0]
    for i in range(1, len(nums)):
        best = min(best, nums[i] - nums[i - 1])
    return best
```

The insight: once sorted (`[1,4,9,13,32]`), you only compare **adjacent** items — the closest pair can't be far apart in the sorted order. Turns an $O(n^2)$ all-pairs check into O(n log n).

---

6. Greedy — grab the best local choice each step

Idea: build the answer by always taking the choice that looks best *right now* (biggest, smallest, closest), without looking ahead. E.g. "minimum number of coins/moves/steps" often means: sort, then repeatedly take the largest that fits.

> **Warning:** greedy is fast but not always correct — the best local move isn't always the best overall. **Test a greedy idea against the given examples before trusting it.** If it fails an example, fall back to another pattern.

Real GCA problem — `maxMeetings(intervals)`: given `(start, end)` times, the most meetings you can attend without overlap. `[(1,3),(2,4),(3,5)]` → 2.

```
def max_meetings(intervals):
    intervals = sorted(intervals, key=lambda x: x[1])   # sort by EARLIEST end
time
    count = 0
    end = float('-inf')                 # when the last chosen meeting finished
    for s, e in intervals:
        if s >= end:                    # this meeting starts after the last one
ended
            count += 1
            end = e                     # take it
    return count
```

The greedy choice: **always take the meeting that ends earliest** — it leaves the most room for the rest. Sorting by end time makes this one clean pass. (This particular greedy IS provably correct, which is why it's a classic.)

Recognition table — phrase → pattern

The problem says...	Try
"seen before / duplicate / count / frequency"	dict or set

The problem says...	Try
"sum of a range / subarray sum"	prefix sums
"sorted array + pair / target"	two pointers
"longest / shortest contiguous substring or subarray"	sliding window
"minimum difference / closest pair / matching"	sort first
"n ≤ 1000"	just brute force

Exercises (in `practice/q4_problem_solving.py`, second half)

1. `max_subarray_sum_k(nums, k)` — max sum of any window of size k (fixed sliding window).
2. `smallest_difference(a, b)` — closest pair between two lists (sort + two pointers).
3. `count_pairs_with_sum(nums, target)` — count pairs adding to target (dict).
4. `equal_split_index(nums)` — index where left sum == right sum (prefix sums).

Worked answers

1. `max_subarray_sum_k(nums, k)` — sliding window (fixed size)

```
def max_subarray_sum_k(nums, k):
    window = sum(nums[:k])
    best = window
    for i in range(k, len(nums)):
        window += nums[i] - nums[i - k]      # slide: add new, drop old
        best = max(best, window)
    return best
```

2. `smallest_difference(a, b)` — sort both, then two pointers

```
def smallest_difference(a, b):
    a, b = sorted(a), sorted(b)
    i = j = 0
    best = abs(a[0] - b[0])
    while i < len(a) and j < len(b):
        best = min(best, abs(a[i] - b[j]))
        if a[i] < b[j]:                       # advance the pointer at the smaller
value
            i += 1
        else:
            j += 1
    return best
```

3. `count_pairs_with_sum(nums, target)` — Two Sum "seen partner", with a count

```
from collections import Counter
def count_pairs_with_sum(nums, target):
    seen = Counter()
    count = 0
    for x in nums:
        count += seen[target - x]            # every earlier partner forms a pair
        seen[x] += 1
    return count
```

4. `equal_split_index(nums)` — prefix sum (left running total)

```
def equal_split_index(nums):
    total = sum(nums)
    left = 0
    for i, x in enumerate(nums):
        right = total - left - x             # everything except left and nums[i]
        if left == right:
            return i
        left += x
    return -1
```

Full runnable versions + more explanation are in `practice/solutions/solutions_q4_explained.md`.